

Surrogate Modeling in Statistical Learning:

A Deep Dive into Gaussian Processes

Prateek Raj (BT23CS021)

Indian Institute of Technology Madras

Research Intern under the guidance of:

Prof. Neelesh Shankar Upadhye

Department of Mathematics

June 2026

1. Introduction & Inductive Bias

- ▶ Supervised Learning Setup
- ▶ The Inductive Ambiguity Problem
- ▶ Restriction Bias vs. Preference Bias
- ▶ Bayes' Rule & Weight Priors

2. Gaussian Process Regression

- ▶ Weight-Space Bayesian Linear Regression
- ▶ Feature Space & Dimension Bottleneck
- ▶ Kernel Trick & Function-Space View
- ▶ Hyperparameter Tuning & Occam's Razor

- ▶ **Definition:** Given training examples of inputs (X) and corresponding outputs (Y), find the underlying relationship:

$$y = f(x)$$

- ▶ **Generalisation:** For a new input x_* , predicting the corresponding target y_* is called generalisation ($f : X \rightarrow Y$).
- ▶ **Problem Types:**
 - **Regression:** Output target is a continuous numerical value (e.g., age, house price, temperature).
 - **Classification:** Output target belongs to a discrete category (e.g., cat/dog, pass/fail).

The Core Problem: Inductive Bias

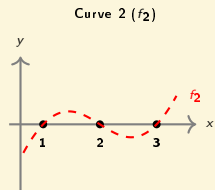
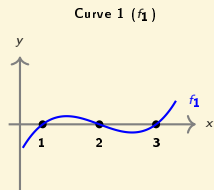
- **Under-determination Problem:** The training data alone cannot uniquely identify the true function. There are infinitely many functions that satisfy the dataset.

Example — Ambiguity in Curve Fitting

Given points $(1, 0)$, $(2, 0)$, $(3, 0)$, possible functions include:

$$f_1(x) = (x - 1)(x - 2)(x - 3)$$

$$f_2(x) = (x - 1)(x - 2)(x - 3) + \sin(x - 1) \sin(x - 2) \sin(x - 3)$$



- **Inductive Bias:** The set of additional assumptions we impose on the learning algorithm to prefer a specific function over others.
- *Every machine learning algorithm contains some form of inductive bias.*

How do we actually impose this bias in our learning algorithm? There are two main strategies:

1. Restriction Bias

Restrict the set of allowed functions. Confine the search space to a rigid, parameterized family of curves.

Model Type: Parametric Models (e.g., Linear functions).

2. Preference Bias

Allow a very large (or infinite) class of functions, but place a probability distribution over them, preferring some over others.

Model Type: Non-Parametric Bayesian Models (e.g., GPs).

- ▶ **Parametric Model:** Look for a solution within a specific family determined by a *finite* number of parameters w .
- ▶ **Simple Linear Model:**

$$f(x) = x^T w, \quad x = [x_1, \dots, x_d]^T, \quad w = [w_1, \dots, w_d]^T$$

- ▶ **Drawback:** The model is strictly prohibited from considering non-linear curves, oscillatory functions, or complex local gradients, leading to under-fitting.

Measurement Noise

The models are never perfect. We model this by adding noise ϵ :

$$y = f(x) + \epsilon$$

Why Gaussian Noise?

- ▶ **Central Limit Theorem:** Sum of many natural random errors approximately converges to a Gaussian.
 - Ref: Jaynes, E. T. (2003). *Probability Theory: The Logic of Science*. Cambridge University Press.
- ▶ **Analytical Convenience:** Easy to manipulate mathematically, yielding closed-form solutions.

Noise Model

We assume i.i.d. additive Gaussian noise:

$$\epsilon_i \sim \mathcal{N}(0, \sigma_n^2), \quad y_i = f(x_i) + \epsilon_i$$

The Likelihood Function

Single Datapoint (x_i, y_i) :

$$p(y_i \mid x_i, w) = \mathcal{N}(x_i^T w, \sigma_n^2)$$

Entire Dataset (X, Y) :

$$p(Y \mid X, w) = \mathcal{N}(Xw, \sigma_n^2 I)$$

Term Notation

$x_i \in \mathbb{R}^d$: Input vector for point i

$X = [x_1, \dots, x_n]^T \in \mathbb{R}^{n \times d}$: Design matrix

$w \in \mathbb{R}^d$: Model weights vector

$y_i \in \mathbb{R}$: Target output for point i

$Y = [y_1, \dots, y_n]^T \in \mathbb{R}^n$: Target vector

σ_n^2 : Noise variance (I is the identity matrix)

Instead of excluding non-linear functions (restriction), we adopt a flexible Bayesian approach:

- ▶ All functions are allowed, but we assign probabilities to them (e.g., simple functions are more likely; highly complex ones are less likely).
- ▶ **Bayes' Rule:**

$$P(w \mid D) = \frac{P(D \mid w) P(w)}{P(D)}$$

Posterior $P(w \mid D)$

Belief about weight parameters *after* observing data.

Prior $P(w)$

Initial belief *before* seeing data. This is where **inductive bias** enters.

Evidence $P(D)$

Normalizing constant (marginal likelihood).

- ▶ We place a zero-mean Gaussian prior on the weight vector:

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \Sigma_p)$$

- ▶ **Prior Mean** $= \mathbf{0}$: Reflects our expectation that smaller weights are more likely. Positive and negative weights cancel out.
- ▶ **Prior Covariance** Σ_p : Determines the dispersion of weights around the prior mean $\mathbf{0}$ (diagonal elements) and the Measures, how two weight vary together (off-diagonal elements).

Combining Prior and Likelihood

Multiplying the likelihood and the prior is proportional to **posterior distribution** over the weights, which is the key object in Bayesian Linear Regression.

- ▶ **Weight-Space View:** The function is expressed in terms of the weight vector w . Instead of finding a single best weight vector, we maintain a *probability distribution* over all possible weights.
- ▶ **Conjugate Property:** Since both the prior and likelihood are Gaussian, the posterior is also Gaussian:

$$p(w \mid X, Y) = \mathcal{N}(\mu_{\text{post}}, \Sigma_{\text{post}})$$

- ▶ **Completing the Square:** We expand the exponent of $p(Y \mid X, w)p(w)$ and match its quadratic and linear weight terms to the standard Gaussian exponent $-\frac{1}{2}(w - \mu)^T \Sigma^{-1}(w - \mu)$:
 - **Quadratic term matching:** $\Sigma_{\text{post}}^{-1} = \Sigma_p^{-1} + \frac{1}{\sigma_n^2} X^T X$
 - **Linear term matching:** $\Sigma_{\text{post}}^{-1} \mu_{\text{post}} = \frac{1}{\sigma_n^2} X^T Y$

$$\Sigma_{\text{post}} = \left(\Sigma_p^{-1} + \frac{1}{\sigma_n^2} X^T X \right)^{-1}, \quad \mu_{\text{post}} = \Sigma_{\text{post}} \frac{1}{\sigma_n^2} X^T Y$$

- ▶ **Classical Regression:** Chooses one weight vector w and predicts $y_* = x_*^T w$, providing no estimation of uncertainty.
- ▶ **Bayesian Regression:** Averages predictions over all possible weights, weighted by their posterior probabilities:

$$p(y_* | x_*, \mathcal{D}) = \int p(y_* | x_*, w) p(w | X, Y) dw$$

- ▶ Evaluating this integral yields:

$$y_* | x_*, \mathcal{D} \sim \mathcal{N}\left(\frac{1}{\sigma_n^2} x_*^T \Sigma_{\text{post}} X^T Y, x_*^T \Sigma_{\text{post}} x_*\right)$$

- ▶ This gives both the **predictive mean** (our estimate) and **predictive variance** (our uncertainty).

- ▶ Since real-world relationships are rarely linear, we transform the input vector $x \in X$ into a high-dimensional feature space using basis functions $\phi(x)$:

$$f(x) = \phi(x)^T w$$

- ▶ The model remains *linear* in terms of weights w , but is now highly *non-linear* in terms of input x .

Polynomial Projection — For a scalar input x :

$$\phi(x) = (1, x, x^2, x^3, \dots)^T$$

Any degree polynomial fit becomes a *linear* model in this feature space.

The Dimension Bottleneck

The Bottleneck ($N \times N$)

If the feature space has dimension N (where N is very large), calculating the posterior covariance is computationally impossible:

$$\Sigma_{\text{post}} = \left(\Sigma_p^{-1} + \frac{1}{\sigma_n^2} \Phi^T \Phi \right)^{-1}$$

This is the **Dimension Bottleneck** of size $N \times N$.

The Woodbury Solution ($n \times n$)

Applying the Woodbury identity:

$$(B + UCV)^{-1} = B^{-1} - B^{-1}U(C^{-1} + VB^{-1}U)^{-1}VB^{-1}$$

Selecting:

$$B = \Sigma_p^{-1}, \quad U = \Phi^T, \quad C = \sigma_n^{-2}I, \quad V = \Phi$$

Rewritten formula as a result:

$$\Sigma_{\text{post}} = \Sigma_p - \Sigma_p \Phi^T \left(\Phi \Sigma_p \Phi^T + \sigma_n^2 I \right)^{-1} \Phi \Sigma_p$$

This allows us to invert a matrix of size $n \times n$ (data points) instead of $N \times N$ (features).